

THESIS FINAL YEAR PROJECT REPORT

Presented to obtain the degree of
NATIONAL ENGINEERING DIPLOMA
Specialization: ELECTRICAL ENGINEERING

Presented By:
NAME & FAMILY-NAME
Born on: XX/XX/XXXX in Monastir

Internship Project TITLE

Defended on 18/07/2026, Before the examination committee:

Mr. Chairman's name	Chair
Mr. Member's name	Member
Mr. supervisor-1 name	Academic Supervisor
Mr. supervisor-2 name	Industrial Supervisor

Dedication

Use this optional page for a short personal dedication. Mention the people, values, or support that made the work meaningful to you.

Keep the tone respectful and concise. Avoid long biographical details, private stories that do not belong in an academic report, or acknowledgements that should appear on the next page.

Acknowledgements

Thank the people and organizations that directly supported the project: academic supervisors, industrial supervisors, team members, laboratory staff, colleagues, family, or funding bodies when relevant.

Be specific about the type of support received, such as guidance, technical feedback, access to resources, review time, or administrative help. Avoid exaggeration, informal jokes, excessive personal detail, and claims that sound like project results.

Abstract

Write the abstract after the report is complete. In one compact paragraph, state the problem, the objective, the method, the main results, and the conclusion. A reader should understand what was done and why it matters without reading the full report.

Avoid citations, undefined acronyms, implementation trivia, broad historical background, and promises that are not supported by the validation chapter.

Keep the abstract factual and proportional to the evidence presented in the report.

Keywords: keyword one; keyword two; keyword three; keyword four.

Contents

Dedication	i
Acknowledgements	ii
Abstract	iii
List of Abbreviations and Nomenclature	viii
General Introduction	1
1 Project Context	2
1.1 Internship Subject and Application Context	2
1.2 Host Enterprise Presentation	2
1.3 Academic and Industrial Context	3
1.4 System Overview	3
1.5 Stakeholder Needs	3
1.6 Project Scope and Constraints	3
1.7 Conclusion	3
2 State of the Art and Literature Review	4
2.1 Theoretical Background	4
2.2 Existing Methods and Technologies	4
2.3 Related Systems	5
2.4 Identified Gaps	5
2.5 Conclusion	5
3 Requirements, Specifications, and Evaluation Protocol	6
3.1 Non-Functional Requirements	6
3.2 Functional Requirements	6
3.3 Technical Constraints	7
3.4 Success Criteria	7
3.5 Evaluation Protocol	7
3.6 Chapter Summary	7
4 Methodology and Proposed Approach	8
4.1 Engineering Approach	8
4.2 Design Logic	8
4.3 Tools and Technologies	9
4.4 Justification of Choices	9
4.5 Conclusion	9

5	Analysis and Design	10
5.1	Overall Architecture	10
5.2	Subsystem Decomposition	10
5.3	Data Flow and Control Flow	11
5.4	Design Decisions	11
5.5	Design Diagrams	11
5.6	Conclusion	11
6	Implementation and Realization	12
6.1	Development Environment	12
6.2	Implementation Details	12
6.3	Software and Hardware Integration	13
6.4	Key Implementation Choices	13
6.5	Conclusion	13
7	Validation, Testing, and Results	14
7.1	Validation Strategy	14
7.2	Test Setup	14
7.3	Experiments	14
7.4	Results	15
7.5	Requirement Traceability Matrix	15
7.6	Comparison with Requirements	15
7.7	Threats to Validity	15
7.8	Conclusion	15
8	Discussion	16
8.1	Interpretation of Results	16
8.2	Limitations	16
8.3	Lessons Learned	16
8.4	Possible Improvements	17
8.5	Conclusion	17
	General Conclusion	18
A	Supporting Material	19
A.1	Supporting Tables or Details	19
B	Additional Results	20
B.1	Detailed Evidence	20
B.2	Evidence Selection Rules	20
B.3	Artifact Tables	20
B.4	Evidence Boundary Summary	21
	References	22

List of Figures

1.1	Project context placeholder	2
2.1	Literature map placeholder	4
3.1	Evaluation protocol placeholder	7
4.1	Methodology workflow placeholder	8
5.1	Overall architecture placeholder	10
6.1	Implementation flow placeholder	12
7.1	Test setup placeholder	14
8.1	Limitations map placeholder	16
A.1	Supporting material placeholder	19
B.1	Additional results placeholder	20

List of Tables

1.1	Context summary template	3
2.1	Comparison matrix template	5
3.1	Non-functional requirements template	6
3.2	Functional requirements template	6
4.1	Choice justification template	9
5.1	Subsystem decomposition template	11
6.1	Implementation choices template	13
7.1	Results summary template	15
7.2	Requirement traceability matrix template	15
A.1	Supporting material register template	19
B.1	Additional artifact template	21

List of Abbreviations and Nomenclature

List only abbreviations, symbols, and technical terms that are used in the report. Define each entry once and use the same wording consistently in later chapters.

Avoid filling this page with common words or terms that never appear in the text. Remove the example entries below before submission.

API Application Programming Interface.

FR Functional Requirement.

NFR Non-Functional Requirement.

UI User Interface.

<TERM> Replace with a project-specific abbreviation or symbol.

General Introduction

Use the general introduction to lead the reader from the broad domain to the precise report problem. Start with the real context, explain the gap or need, then state the project objective and the structure of the report.

Say what problem exists, who is affected by it, why it matters now, and what contribution your work intends to make. Keep the opening connected to the rest of the report, especially the requirements, methodology, and validation chapters.

Avoid beginning with vague claims such as “technology is everywhere” unless they quickly become specific. Do not present results before the validation chapter, do not overpromise the scope, and do not include company history that belongs in the project-context chapter.

The final paragraph should announce the organization of the report in the same order as the chapters. Keep this roadmap short and factual.

Chapter 1

Project Context

This chapter frames the project before technical details begin. Present the host environment, the problem setting, the stakeholders, and the boundaries of the work.

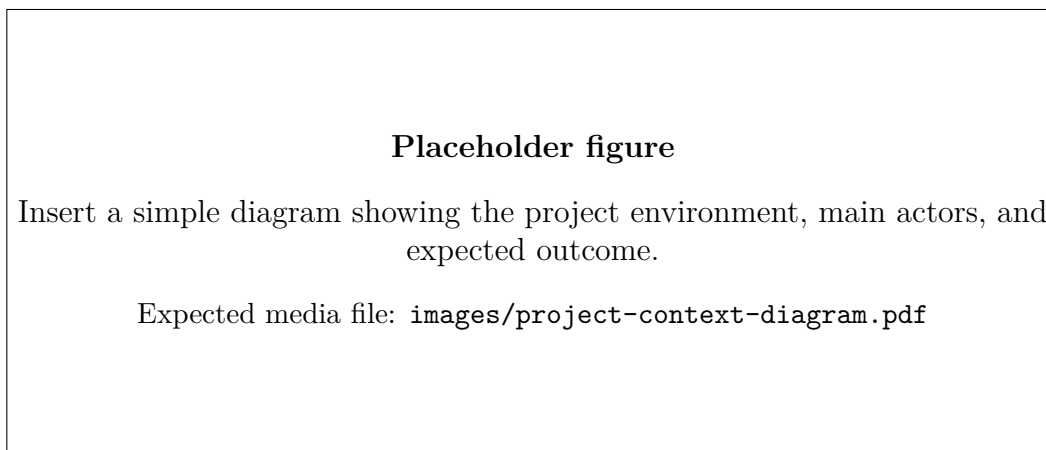


Figure 1.1: Project context placeholder

1.1 Internship Subject and Application Context

Explain the subject in practical terms: the situation before the project, the need that motivated the work, and the application domain in which the solution will be used.

Focus on facts that help the reader understand the project. Avoid turning this section into a technical design description or a copy of the internship offer.

1.2 Host Enterprise Presentation

Present the host organization briefly: its domain, activities, relevant department or team, and why this environment is connected to the project.

Mention only information that supports the report context. Avoid promotional language, long corporate history, confidential details, or large catalogs of products and customers.

1.3 Academic and Industrial Context

Clarify the academic interest and practical value of the project. Connect the topic to engineering knowledge, research questions, professional practice, or operational constraints.

Avoid claiming scientific novelty without evidence. If the work is mainly applied engineering, say so clearly and explain its value.

1.4 System Overview

Give a high-level view of the existing system, process, or environment. Identify the main components, users, inputs, outputs, and dependencies.

Table 1.1: Context summary template

Element	Guidance
Host environment	State where the work took place and which team or department was involved.
Problem setting	Describe the operational pain point or opportunity addressed by the project.
Main users	Identify who will use, review, maintain, or benefit from the result.
Constraints	Mention time, tools, data, hardware, access, standards, or organizational limits.

1.5 Stakeholder Needs

List the needs of users, supervisors, maintainers, customers, reviewers, or other stakeholders. Explain conflicts or trade-offs when they influence later requirements.

Avoid inventing stakeholders only to fill space. Every need introduced here should connect to a requirement, design decision, or validation criterion later.

1.6 Project Scope and Constraints

Define what is included, what is excluded, and what assumptions limit the work. This section protects the report from unrealistic expectations.

Avoid hiding weak points. Clear boundaries make the validation chapter more credible.

1.7 Conclusion

Summarize the context in a few sentences and transition to the state of the art or literature review. Do not introduce new technical details in the conclusion.

Chapter 2

State of the Art and Literature Review

This chapter shows what is already known and where your project fits. It should compare sources, methods, tools, standards, or systems rather than simply list them.

2.1 Theoretical Background

Introduce the concepts required to understand the project. Define the technical vocabulary, models, standards, or scientific principles that will be used later.

Avoid explaining every basic term from the field. Focus on background that directly supports the problem, method, or evaluation.

2.2 Existing Methods and Technologies

Describe the main approaches currently used to solve similar problems. For each approach, explain its principle, strengths, limits, and conditions of use.

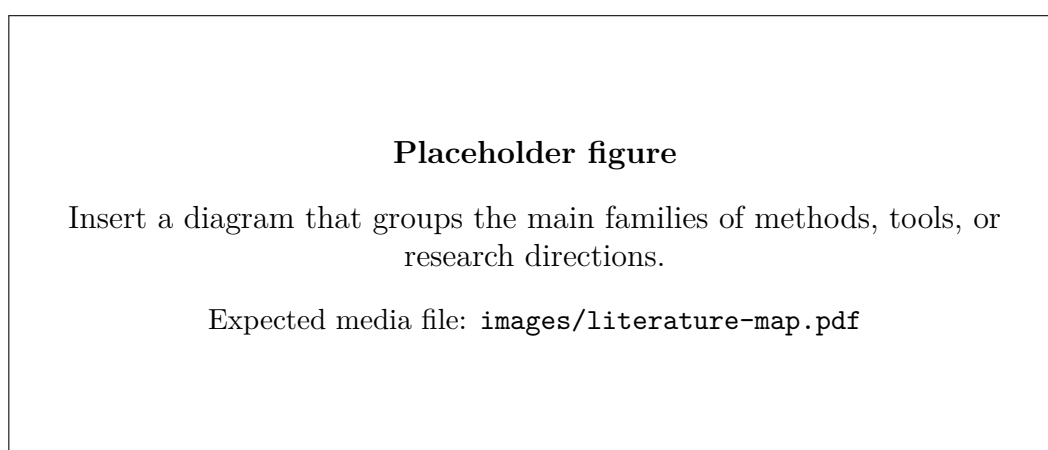


Figure 2.1: Literature map placeholder

2.3 Related Systems

Compare representative systems, tools, frameworks, products, or academic prototypes. Use consistent comparison criteria so the reader can see why your choices are reasonable.

Table 2.1: Comparison matrix template

Approach	Strengths	Limitations	Relevance to this project
Approach A	Replace with evidence-based strengths.	Replace with observed or cited limits.	Explain the lesson retained.
Approach B	Replace with evidence-based strengths.	Replace with observed or cited limits.	Explain the lesson retained.

Avoid unsupported rankings such as “best” or “most powerful” unless the criteria and evidence are explicit.

2.4 Identified Gaps

End the chapter by stating the gap that your project addresses. The gap may be technical, methodological, organizational, experimental, or usability-related.

Avoid forcing a gap that your work does not actually address. A modest, well-supported gap is stronger than an exaggerated claim.

2.5 Conclusion

Summarize the lessons taken from the literature and explain how they shape the requirements or methodology of the next chapters.

Chapter 3

Requirements, Specifications, and Evaluation Protocol

This chapter translates the problem into verifiable expectations. Requirements should be clear enough to guide design and later validation.

3.1 Non-Functional Requirements

Describe quality expectations such as reliability, usability, maintainability, performance, security, scalability, portability, or traceability.

Table 3.1: Non-functional requirements template

ID	Quality attribute	Expected evidence
NFR-01	Replace with a quality attribute.	State how this quality will be checked or observed.
NFR-02	Replace with a quality attribute.	State how this quality will be checked or observed.

Avoid vague requirements such as “the system must be user friendly” without measurable or observable evidence.

3.2 Functional Requirements

List the services, behaviors, or operations the system must provide. Use active, testable wording.

Table 3.2: Functional requirements template

ID	Requirement
FR-01	The system shall replace this sentence with a specific, verifiable behavior.
FR-02	The system shall replace this sentence with a specific, verifiable behavior.

Avoid mixing implementation choices into requirements unless they are mandatory constraints imposed by the context.

3.3 Technical Constraints

State constraints related to tools, data, standards, platforms, deadlines, deployment, hardware, software, access rights, or institutional rules.

Avoid presenting personal preferences as constraints. Explain why each constraint exists and how it affects the design.

3.4 Success Criteria

Define what counts as a successful project. Criteria should connect to the objectives and be checked later in the validation chapter.

3.5 Evaluation Protocol

Explain how each requirement will be evaluated. Name the tests, experiments, reviews, demonstrations, datasets, metrics, or acceptance rules that will be used.

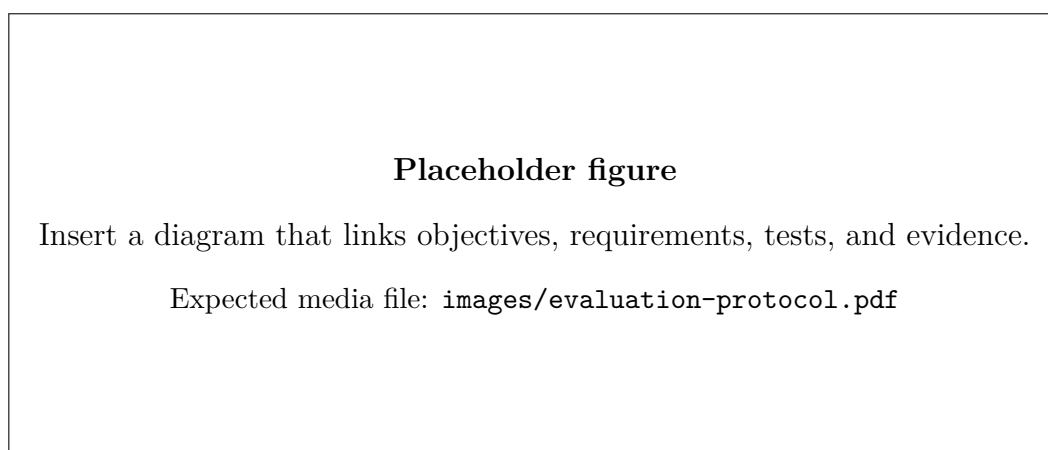


Figure 3.1: Evaluation protocol placeholder

Avoid treating demonstration screenshots as proof unless they are connected to explicit acceptance criteria.

3.6 Chapter Summary

Close by explaining how the requirement baseline guides the methodology and design chapters.

Chapter 4

Methodology and Proposed Approach

This chapter explains how the work is organized to answer the problem. It should justify the approach before describing the detailed design.

4.1 Engineering Approach

Describe the development or research process used in the project: iterative development, experimental protocol, design science, agile workflow, modeling approach, or another suitable method.

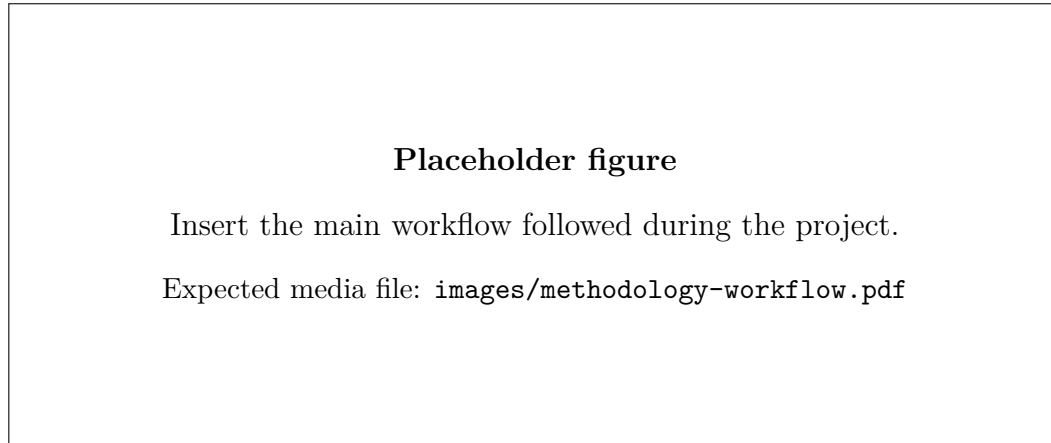


Figure 4.1: Methodology workflow placeholder

Avoid writing a calendar diary. Report the method, not every daily activity.

4.2 Design Logic

Explain the reasoning that connects the requirements to the proposed solution. Show why the chosen approach fits the context and constraints.

Avoid presenting design decisions as arbitrary. Each important choice should answer a need, constraint, risk, or lesson from the literature review.

4.3 Tools and Technologies

List the tools, languages, frameworks, datasets, devices, or platforms used. For each one, state its role in the project.

Avoid tool catalogs. Do not include logos or marketing claims; include only technologies that matter to implementation or validation.

4.4 Justification of Choices

Compare reasonable alternatives and explain why the selected option was retained. Mention trade-offs honestly.

Table 4.1: Choice justification template

Decision	Reason retained	Trade-off or risk
Replace with a choice.	Explain the evidence or constraint.	State the cost, limitation, or mitigation.

4.5 Conclusion

Summarize the adopted methodology and prepare the reader for the analysis and design chapter.

Chapter 5

Analysis and Design

This chapter turns requirements into a structured solution. It should make the system understandable before implementation details appear.

5.1 Overall Architecture

Present the global architecture, main modules, external systems, data sources, users, and communication paths.

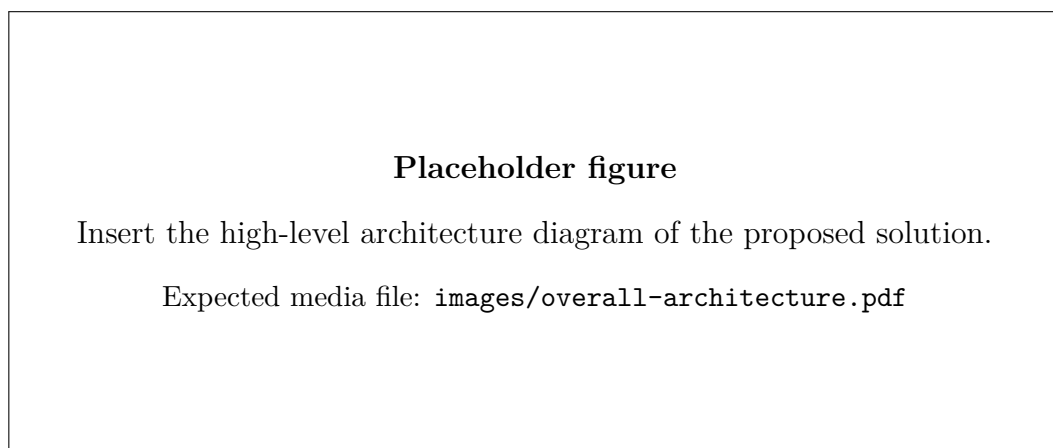


Figure 5.1: Overall architecture placeholder

Avoid drawing every class or function in the global architecture. Keep this view readable.

5.2 Subsystem Decomposition

Break the solution into subsystems or modules. For each one, state its responsibility, inputs, outputs, and interactions.

Table 5.1: Subsystem decomposition template

Subsystem	Responsibility	Main interactions
Subsystem A	Replace with one clear responsibility.	Replace with inputs, outputs, and dependencies.
Subsystem B	Replace with one clear responsibility.	Replace with inputs, outputs, and dependencies.

5.3 Data Flow and Control Flow

Explain how information moves through the system and how decisions or operations are triggered.

Avoid mixing normal flows, error flows, and exceptional cases in one unreadable diagram. Split diagrams when needed.

5.4 Design Decisions

Record the important choices made during design, especially those that affect maintainability, performance, correctness, cost, or user experience.

Avoid documenting trivial choices. Focus on decisions a future maintainer or evaluator would need to understand.

5.5 Design Diagrams

Include UML diagrams, database schemas, sequence diagrams, deployment diagrams, wireframes, or other views that clarify the solution.

Use each diagram in the text: introduce it, explain what it shows, and refer to the design decision it supports.

5.6 Conclusion

Conclude by summarizing the selected design and preparing the transition to implementation.

Chapter 6

Implementation and Realization

This chapter explains how the design was built. It should contain enough technical detail for evaluation and maintenance, without becoming a full code listing.

6.1 Development Environment

Describe the environment needed to reproduce the work: operating system, tools, languages, dependencies, versions, devices, services, and configuration notes.

Avoid assuming the reader has your machine. Mention setup constraints and reproducibility steps where they matter.

6.2 Implementation Details

Present the main implementation units and explain the logic behind them. Use short code snippets only when they clarify an important mechanism.

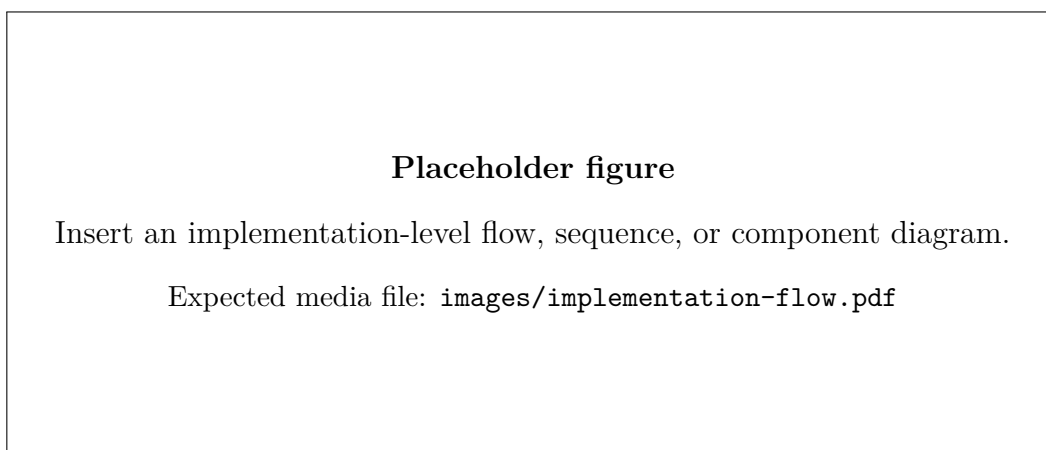


Figure 6.1: Implementation flow placeholder

Avoid pasting large blocks of code. Cite the role of the code, explain its behavior, and keep full listings for appendices if required.

6.3 Software and Hardware Integration

If the project combines software, hardware, services, databases, APIs, or external platforms, explain how they are integrated and what assumptions are required.

Avoid hiding integration problems. Mention adapters, interfaces, protocols, configuration files, and known limits.

6.4 Key Implementation Choices

Explain choices that shaped the implementation: data structures, algorithms, libraries, architecture patterns, error handling, security controls, or deployment decisions.

Table 6.1: Implementation choices template

Choice	Reason	Effect on the project
Replace with a key choice.	Explain the technical reason.	State the benefit, limit, or risk.

6.5 Conclusion

Summarize what was implemented and transition to validation. Do not claim success here; success belongs in the testing and results chapter.

Chapter 7

Validation, Testing, and Results

This chapter proves how far the work satisfies its objectives. It should separate evidence from interpretation.

7.1 Validation Strategy

State what must be validated and why. Connect the strategy to the requirements and success criteria defined earlier.

Avoid saying that the system was “tested successfully” without explaining the test basis, inputs, expected results, and observed results.

7.2 Test Setup

Describe the test environment, datasets, tools, devices, versions, configurations, and assumptions. Include enough detail to make the results reproducible.

7.3 Experiments

Describe each experiment or test campaign. Give the objective, input data, procedure, and acceptance criteria.

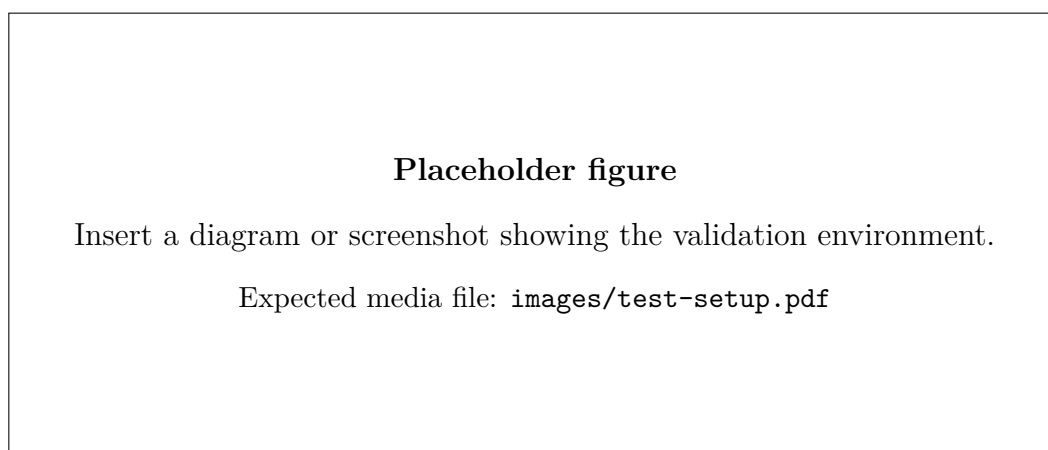


Figure 7.1: Test setup placeholder

7.4 Results

Present measured or observed results in tables, charts, or short explanations. Report failures and partial results as clearly as successful ones.

Table 7.1: Results summary template

Test	Expected result	Observed result	Status
Test case A	Replace with expectation.	Replace with observation.	Pass, fail, partial, or blocked.
Test case B	Replace with expectation.	Replace with observation.	Pass, fail, partial, or blocked.

7.5 Requirement Traceability Matrix

Map each requirement to evidence. This makes it clear which objectives are satisfied and which remain open.

Table 7.2: Requirement traceability matrix template

ID	Evidence	Assessment
FR-01	Replace with test, artifact, review, or measurement.	Satisfied, partial, blocked, or future work.
NFR-01	Replace with test, artifact, review, or measurement.	Satisfied, partial, blocked, or future work.

7.6 Comparison with Requirements

Discuss the results against the original requirements. Highlight mismatches, incomplete evidence, and reasons for deviation.

7.7 Threats to Validity

Identify factors that may limit the credibility or generalization of the results: sample size, test coverage, measurement bias, missing data, hardware limits, user bias, or uncontrolled variables.

Avoid treating limitations as a weakness to hide. Honest validity analysis makes the report stronger.

7.8 Conclusion

Summarize the validation status and prepare the reader for the discussion chapter.

Chapter 8

Discussion

This chapter interprets the results. It explains what the evidence means, why some choices worked, and where the project remains limited.

8.1 Interpretation of Results

Explain the meaning of the validation results in relation to the problem statement. Distinguish confirmed findings from assumptions.

Avoid repeating the results table line by line. Focus on patterns, causes, and consequences.

8.2 Limitations

State the limits of the solution, method, data, implementation, or evaluation. Explain whether each limitation affects correctness, usability, scalability, generality, or future adoption.

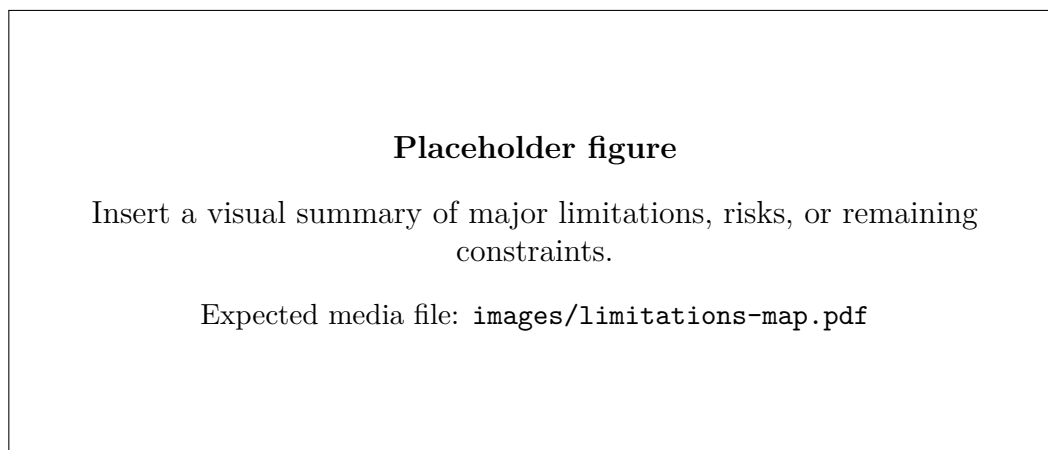


Figure 8.1: Limitations map placeholder

8.3 Lessons Learned

Summarize practical and technical lessons from the project. These may concern methodology, design, tools, communication, data quality, testing, or project management.

Avoid personal diary entries. Keep lessons connected to engineering practice and report evidence.

8.4 Possible Improvements

Propose improvements that logically follow from the limitations and results. Prioritize them by feasibility or impact when possible.

8.5 Conclusion

Close the discussion by stating the main interpretation of the work and its remaining open questions.

General Conclusion

The general conclusion closes the report by returning to the initial problem, objective, methodology, and validated results. It should be short, balanced, and evidence-based.

Begin by restating the problem and the project objective in one paragraph. Then summarize the main work performed, the most important results, and the degree to which the objectives were achieved.

Mention limitations honestly and connect them to future work. Future work should be realistic: propose extensions, deeper validation, deployment improvements, broader experiments, or usability enhancements that follow from the report.

Avoid introducing new technical material, new citations, new results, or emotional closing statements. The conclusion should leave the reader with a clear understanding of what was achieved and what remains to be done.

Appendix A

Supporting Material

Use this appendix for material that supports the report but would interrupt the main chapters: extended tables, configuration details, questionnaires, additional diagrams, or sample inputs.

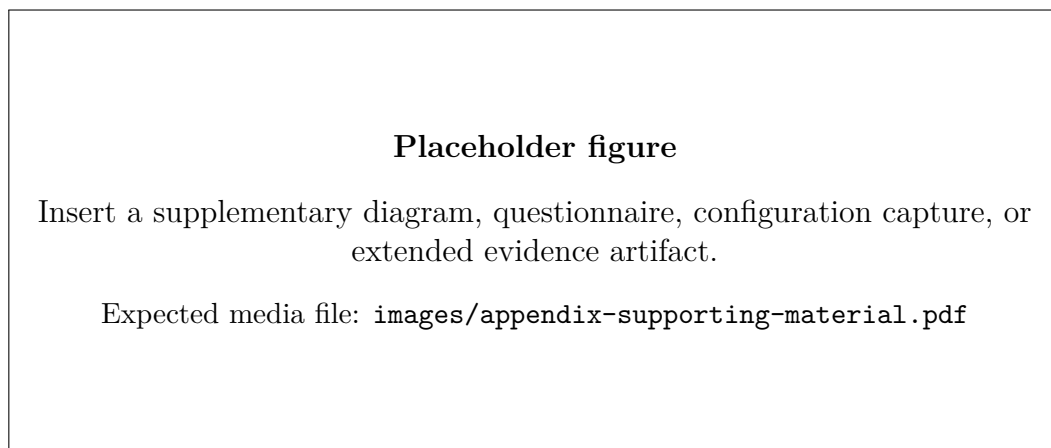


Figure A.1: Supporting material placeholder

A.1 Supporting Tables or Details

Place long but useful material here. Refer to this appendix from the relevant chapter so the reader knows why it matters.

Table A.1: Supporting material register template

Item	Purpose	Referenced from
Replace with item name.	Explain why it supports the report.	Chapter or section.

Avoid using appendices as a storage area for unrelated files. Every appendix item should support a claim, method, or result in the main report.

Appendix B

Additional Results

Use this appendix for secondary results, detailed measurements, extra screenshots, extended logs, or examples that are useful but not essential to the main argument.

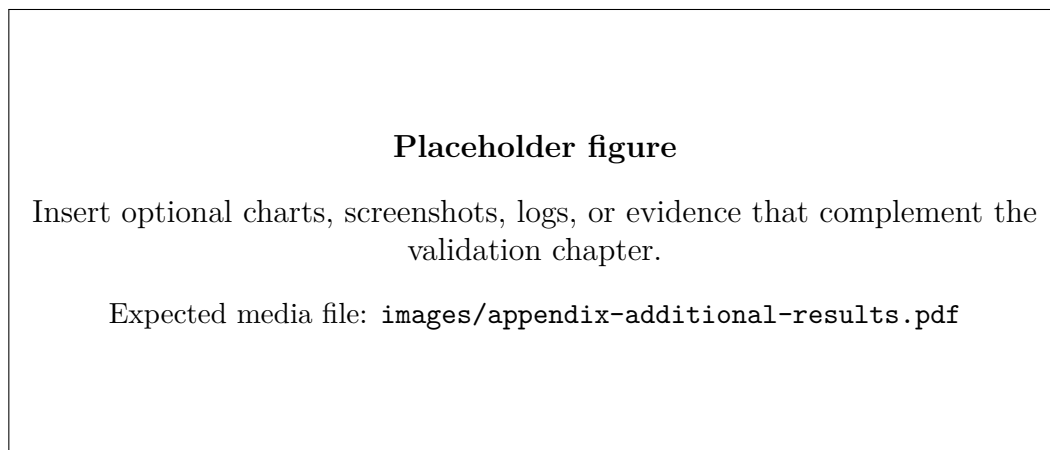


Figure B.1: Additional results placeholder

B.1 Detailed Evidence

Organize additional evidence by test, feature, scenario, dataset, or requirement. State how each item relates to the validation chapter.

B.2 Evidence Selection Rules

Explain why these results were moved to the appendix. Good reasons include length, repetition, secondary importance, or the need to keep the main chapter readable.

Avoid placing weak or contradictory results here to hide them. Important failures and limitations should still be discussed in the main report.

B.3 Artifact Tables

Use tables to list extra artifacts when needed.

Table B.1: Additional artifact template

Artifact	Description	Status
Replace with artifact name.	Explain what the artifact shows.	Complete, partial, or example.

B.4 Evidence Boundary Summary

End by stating what the appendix evidence supports and what it does not prove.

Bibliography

- [1] Author Name, *Book or Report Title*, Publisher, Year.
- [2] Author Name and Author Name, “Article Title,” *Journal or Conference Name*, volume, number, pages, Year.
- [3] Organization Name, *Web Page or Documentation Title*,
<https://example.com>.
Accessed: Month Day, Year.